

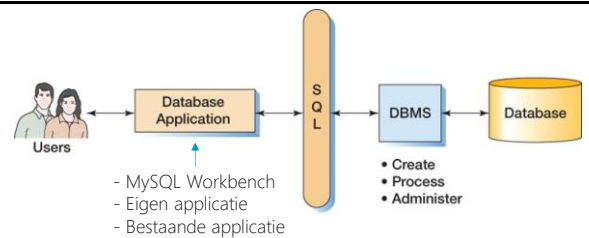
— Databases

Hoofdstuk 10: Inleiding security

L. Espeel



— Koppeling met MySQL



In plaats van MySQL Workbench te gebruiken, dat bedoeld is om databanken te beheren en SQL-queries uit te testen, worden SQL-queries in een echte applicatie geïntegreerd in de back-end code. Een (web)applicatie gebruikt daarbij een gebruiksvriendelijke front-end, terwijl de SQL-queries onzichtbaar op de achtergrond worden uitgevoerd.

- Bv. een formulier met invulvelden en een verzendknop stuurt achterliggend een SQL INSERT-query naar de MySQL-databank.

Veel programmeertalen ondersteunen MySQL-connecties waaronder C#.

Zie o.a.:

- <https://zetcode.com/csharp/mysql>
- <https://dev.mysql.com/doc/connector-net/en/connector-net-introduction.html>

— MySQL-connectiestring

Ongeacht de gebruikte programmeertaal heb je een MySQL-connectiestring nodig met parameters zoals host, gebruiker, wachtwoord en databank.

```
In C#:  
using MySql.Data.MySqlClient;  
  
string connectionString = @"server=localhost;userid=root;password=ww;database=jeugdvereniging";  
MySqlConnection connection = new MySqlConnection(connectionString);  
connection.Open();  
  
// ... MySqlCommand-object met SQL-queries ...
```

Het is echter **nooit** een goed idee om de **root-gebruiker** te gebruiken in applicatiecode (zoals hierboven), omdat deze volledige DDL- en DML-rechten heeft!

3

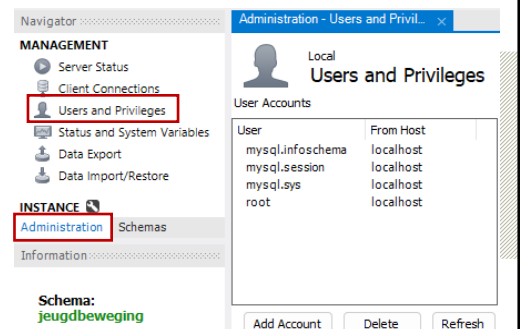
— MySQL-gebruikers

MySQL-gebruikers met hun rechten opvragen:

```
SELECT *  
FROM mysql.user;
```

Host	User	Select_priv	Insert_priv	Update_priv	Delete_priv	Create_priv	Drop_priv
localhost	mysql.infoschema	Y	N	N	N	N	N
localhost	mysql.session	N	N	N	N	N	N
localhost	mysql.sys	N	N	N	N	N	N
localhost	root	Y	Y	Y	Y	Y	Y

Dit kan ook bekeken en beheerd worden in MySQL Workbench:



4

— MySQL-gebruiker

Het is sterk aanbevolen om een aparte MySQL-gebruiker aan te maken met **enkel de rechten die de applicatie nodig heeft**.

1. MySQL-gebruiker aanmaken

```
CREATE USER "appuser"@"localhost" IDENTIFIED BY "sterkWachtwoord123!";
```

 - Als de applicatie niet op hetzelfde toestel staat als de MySQL-server, vervang *localhost* door *%*.
2. Rechten toekennen voor een basis CRUD-applicatie

```
GRANT SELECT, INSERT, UPDATE, DELETE ON jeugdbeweging.* TO "appuser"@"localhost";
```

 - Dit is een **DCL-query (Data Control Language)**.
 - Gebruik *** voor alle tabellen of vervang dit door een specifieke tabelnaam.
3. Ingestelde rechten toepassen

```
FLUSH PRIVILEGES;
```

5

— Wachtwoorden opslaan ⁽¹⁾

De wachtwoorden van MySQL-gebruikers worden niet in plaintext opgeslagen, maar gehasht:

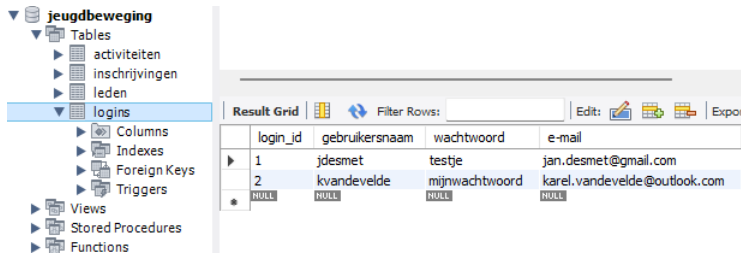
```
SELECT host, user, plugin, authentication_string  
FROM mysql.user;
```

host	user	plugin	authentication_string
localhost	appuser	caching_sha2_password	\$A\$005\$R□□+□leasqp/h~□.3?cGX3yef2hnm...
localhost	mysql.infoschema	caching_sha2_password	\$A\$005\$THISISACOMBINATIONOFINVALIDSAL...
localhost	mysql.session	caching_sha2_password	\$A\$005\$THISISACOMBINATIONOFINVALIDSAL...
localhost	mysql.sys	caching_sha2_password	\$A\$005\$THISISACOMBINATIONOFINVALIDSAL...
localhost	root	caching_sha2_password	\$A\$005\$/□□Z0n;□;{7L7□ t_GHJxToTg1KTD...

6

Wachtwoorden opslaan (2)

Wanneer je in jouw C#-applicatie logins wilt bijhouden, sla je wachtwoorden beter nooit in plaintext op (zoals hieronder het geval is)!



login_id	gebruikersnaam	wachtwoord	e-mail
1	jdesmet	testje	jan.desmet@gmail.com
2	kvandeveld	mijnwachtwoord	karel.vandeveld@outlook.com

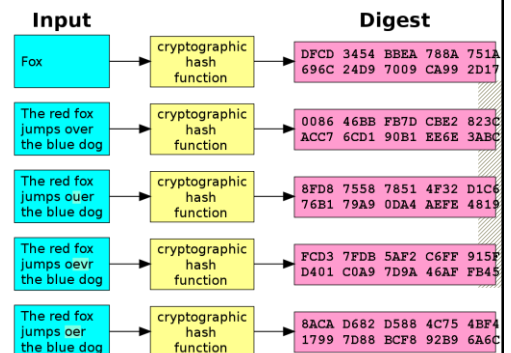
Voorzie dus een onleesbare waarde via **encryptie** of via **hashing**. Hashing is meer aangewezen voor het opslaan van wachtwoorden.

Hashing (1)

Is het proces waarbij we een gegeven waarde converteren naar een andere waarde door gebruik te maken van een hash-functie die bestaat uit een wiskundig algoritme.

De originele waarde (input) noemen we **plaintext**, de output van een hash-functie noemen we een **hash-waarde**.

Het beste aan hash-functies is dat ze slechts in één richting (horen te) werken. De hash-waarde kan zo nooit terug naar de originele waarde.
Dit is het voornaamste verschil in vergelijking met encryptie.



— Hashing (2)

Probleem:

Als we wachtwoorden gehasht opslaan, hoe kunnen we die dan gebruiken bij de authenticatie van een gebruiker?

Een hash-algoritme genereert voor hetzelfde wachtwoord altijd dezelfde hash.

- Als jouw wachtwoord bv. *LiefHondje1234* is, dan kan de hash eruitzien als bv. *s5dfzef98gq79ze6qg54qsgzg* en is dit hetgeen wat je opslaat in de tabel.
- Wanneer je probeert in te loggen en het wachtwoord *liefhondje4567* ingeeft, dan genereert het hash-algoritme bv. *qsdhfef98gdvgbe56qg55daz8g*.

Aangezien beide hashes niet overeenkomen, weet het systeem dat het ingevoerde wachtwoord ongeldig is en kan er een loginfout aan de gebruiker worden teruggegeven.

9

— Belang van het hashing-algoritme (1)

- Het is niet meer gegarandeerd dat elk hashing algoritme jou voldoende zal beschermen tegen aanvallen.
- Zo is er het MD5 hashing algoritme, wat een supersnel hashing algoritme is. Het nadeel is dat een hacker tegenwoordig zeer weinig tijd nodig heeft om de input data te *brute forcen*.
- Daarenboven vind je van veel hashing-algoritmes gigantische dictionaries online waar alle meest voorkomende wachtwoorden werden ingegeven.
Voer de onderstaande Python code maar eens uit en ga naar de website <https://md5decrypt.net> om jouw hash output te decrypteren.

```
1 import hashlib
2 m = hashlib.md5(b"password")
3 print(m.hexdigest())
```

10

— Belang van het hashing-algoritme (2)

- **Niet** alle hashing algoritmes zijn even eenvoudig te kraken. Betere optie is bv. **SHA-2** die bestaat uit meerdere versies waaronder **SHA-256**.
- Belangrijk om te weten is dat een hashingalgoritme altijd een output van vaste lengte genereert. Voor SHA-256 (Secure Hash Algorithm) is de hash bijvoorbeeld altijd 64 hexadecimale karakters lang. Het voornaamste doel van hashing is ervoor te zorgen dat hackers, die eventueel de logintabel in handen krijgen, niet eenvoudig de oorspronkelijke inputwaarden van de wachtwoorden kunnen achterhalen.
- Omdat een hashfunctie slechts een beperkt aantal mogelijke outputwaarden heeft, bestaat de kans dat twee verschillende inputwaarden dezelfde hash genereren. Dat noemen we een **collisie**. Bij een zwak hash-algoritme zoals MD5 komt dit relatief vaak voor; bij SHA-256 is de kans kleiner.
- Toch kunnen we concluderen dat hashing op zichzelf onvoldoende is voor veilige wachtwoordopslag. We kunnen nog extra zaken toevoegen zoals **salt**ing en **pepper**ing.

11

— Salt

```
//a plain text string
p@55word

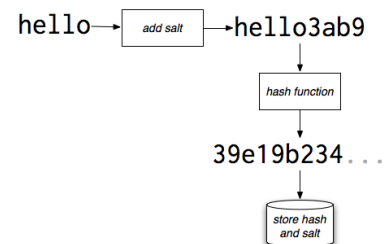
//salt
ABCD123

//combined string
p@55wordABCD123

//hashed string (using SHA256):
d30bad8c6885624968f264dc86e063b0fe57e66354d3d59d695e56cc24cb83ca
```

- In cryptografie is *salt* een methode om random data toe te voegen aan de input van de hash-functie. Er wordt een nieuwe salt gegenereerd voor elke input data.
- Het gehasht wachtwoord en de salt worden samen opgeslagen in de databank.
- Voorkomt **rainbow table-aanvallen** (zoek zelf eens op wat dit is ...).

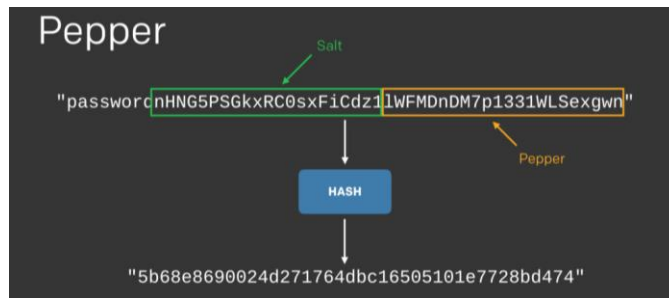
				
Password	p4s5w3rdz	p4s5w3rdz	p4s5w3rdz	p4s5w3rdz
Salt	-	-	et52ed	ye5sf8
Hash	f4c31aa	f4c31aa	1vn49sa	z32i6t0



12

— Pepper

- Is gelijkaardig aan salt. Ook hier wordt een random string toegevoegd aan de input-waarde van een hash-functie.
- In de plaats dat we de *pepper* opslaan in een databank, houden we het in dit geval echter geheim. Ofwel slaan we de pepper-waarde op in een apart medium ofwel helemaal niet.
- Voorkomt **database compromise aanvallen** (zoek zelf eens op wat dit is ...).



13

— Case: login-applicatie

Stel dat we een grafische C#-applicatie hebben gekoppeld aan de MySQL-databank met de mogelijkheid om een gebruiker te registreren en in te loggen:

- De applicatie bevat een registratieformulier om een nieuwe gebruiker aan te maken met o.a. zijn wachtwoord.
Achter de knop "Registreren" bevindt zich de code met een *INSERT*-query om de gegevens in de tabel *logins* op te slaan. Het wachtwoord moet daarbij al gehasht worden meegegeven.
- De applicatie bevat een loginformulier waarin de gebruiker zijn gebruikersnaam en wachtwoord invoert. Achter de knop "Inloggen" bevindt zich de code met een *SELECT*-query om de gegevens zoals het gehasht wachtwoord van de gebruiker uit de tabel *logins* op te halen volgens zijn gebruikersnaam.

De volgende slides tonen C# code-snippets voor het opslaan en verifiëren van gehashte wachtwoorden.

14

— Code: registratie gebruiker (1)

1. Genereer een willekeurige salt

```
//Generate a random string of 20 characters (Salt)
string Salt = GetRandomString(20);

//GetRandomString function
// (for reference only, you can use any method to generate random string)
public string GetRandomString(int length)
{
    //create random class instance
    Random random = new Random();

    //all characters and numbers as input
    string chars = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";

    //generate random string for the supplied length argument
    string str = new string(Enumerable.Repeat(chars, length)
        .Select(s => s[random.Next(s.Length)])
        .ToArray());

    return str;
}
```

15

hogeschool
vives

— Code: registratie gebruiker (2)

2. Bereken de hash-waarde

Plaintext, afkomstig van het invulveld.

```
//Generate hash of password + salt
string hashedPassword = GetHashedPassword>Password + Salt)

public string GetHashedPassword(string input)
{
    //create cryptography class instance
    SHA256 sha256 = SHA256.Create();

    //compute bytes of input string
    byte[] bytes = sha256.ComputeHash(Encoding.Unicode.GetBytes(input));

    //compute string from bytes
    StringBuilder builder = new StringBuilder();
    for (int i = 0; i < bytes.Length; i++)
    {
        builder.Append(bytes[i].ToString("x2"));
    }

    //return hashed string
    return builder.ToString();
}
```

16

hogeschool
vives

— Code: inloggen gebruiker

3. Vergelijk de hash-waarde van het opgegeven wachtwoord met de hash-waarde van het opgeslagen wachtwoord

```
string username = <from login page>
string password = <from login page>

//find user record based on username
//-- your code here

//if user not found, it's invalid Login attempt

//retrieve user's Hashed Password and Salt
string hashedPassword = <from user record>
string salt = <from user record>

//compute hash based on password entered by user and salt stored with us
string hashedInput = GetHashedPassword(password + salt)

//compare hashedInput with hashedPassword
if (hashedInput != hashedPassword)
{
    //invalid Login ... your code to return error here
}
else
{
    //valid Login .. your code may continue to execute
}
```